Interactive Arcade System with Physical Tile Grid

1. Design Intent

A full-scale arcade installation that transforms digital games into physical experiences. Players step onto a 3×3 grid of illuminated floor tiles, surrounded by overhead disco lighting effects, laser mood lighting and gaming, and motorized visual indicators. Each game (DDR-style rhythm, Memory, Whack-a-Mole, Tic-Tac-Toe) triggers synchronized lighting patterns and motor sequences that match the gameplay intensity and state.

The system creates a gaming experience where every move is reflected in light, color, and motion. Overhead effects telegraph game state to spectators, turning solo gameplay into a shared visual spectacle. The tile buttons have the perfect resistance.

Who is it for?

- Installation/entertainment spaces seeking interactive exhibits
- Maker faires and technology showcases
- Home game rooms with dedicated space
- Interactive museums and science centers
- Event spaces wanting memorable experiences

What problem does it solve?

Reduces the hardware and spacial requirements to have an arcade library.

2. Inspiration & References

1. **Dance Dance Revolution (DDR) arcade cabinets**
2. **Memory Game**
3. **Whack a Mole**
4. **Tic Tac Toe**

Visual References:

- DDR arcade cabinet layout (player positioning, screen placement

-

- Laser shows

3. User Journey

1. **User approaches arcade floor on attract mode**

    - Overhead disco scrolls pink/purple/blue gradient bands
    - Laser-Stepper combos create laser show
    - Floor tiles are dark, center tile pulses subtly
2. **User steps on CENTER tile**

    - System wakes up
    - Disco shifts to cyan/blue alternating pattern
    - Browser displays main menu (DDR selected by default)
    - Arrow tiles light up: LEFT/RIGHT to navigate, CENTER to confirm
3. **User steps LEFT or RIGHT arrows**

    - Menu carousel updates

    - Disco pattern changes instantly:
        - DDR: cyan/blue swap
        - Memory: orange/warm-orange swap
        - Whack-a-Mole: green-light/green-dark swap
        - Tic-Tac-Toe: magenta/pink-light swap
4. **User steps CENTER to confirm**

    - Instructions are displayed for 5 seconds
    - Countdown begins (3...2...1...)
    - Disco changes to game-specific pattern
    - Tiles light up with game-specific colors
5. **Gameplay active (DDR example)**

    - User steps on lit tiles in rhythm
    - Correct step: tile changes color, disco scrolls faster
    - Wrong step: both steppers spin opposite directions (0.5s), tile flashes red
    - IR beam break: corresponding stepper spins 1 revolution, bonus points awarded
6. **Gameplay active (Memory example)**

    - Watch phase: disco warm/dim pattern, tiles flash sequence
    - Play phase: disco scrolls blue/green gradient, user repeats sequence
    - Wrong tile: steppers spin opposite, game resets
7. **Gameplay active (Whack-a-Mole example)**

    - Disco scrolls green/cyan/orange/pink/purple/yellow
    - Tiles light up randomly
    - User steps on lit tile: success feedback, disco continues
    - Miss: steppers spin briefly
8. **Gameplay active (Tic-Tac-Toe example)**

    - Player pink vs player purple
    - Disco scrolls pink gradient during pink turn
    - Disco scrolls purple gradient during purple turn
    - Turn change: corresponding stepper spins 1.5 revolutions
9. **Game ends — WIN**

    - Celebration pattern: disco scrolls rainbow at high speed
    - Steppers alternate: both CW 2 revs, then both CCW 2 revs (loop)
    - End screen displays score
10. **Game ends — LOSE**

    - Failure pattern: disco alternates red/dark-red at slow pulse
    - Steppers stop
    - End screen displays "Try Again"

11. **60 seconds idle**
    - System returns to attract mode
    - Disco resumes pink/purple/blue scroll
    - Steppers resume slow alternating spins

4. Success Criteria

Functional Requirements:

Metric	Target	Actual
Tile button response time	<50ms	~30ms (debounced polling)
NeoPixel frame rate	50fps (20ms)	50fps stable
Stepper motor smoothness	No visible stutter	Background thread achieves
Tile button debounce	Prevent double-triggers	500ms window
IR sensor debounce	Prevent double-triggers	500ms window
HTTP polling latency	<50ms round-trip	~30-40ms typical
WiFi reconnection	Automatic recovery	Floor AP stable
Highly durable tile buttons	Retains elasticity at 120kg+ weight	No physical sensor, sturdy flexible materials.

User Experience:

- Color-coded game modes instantly recognizable
- Stepper movements feel intentional, not random
- Disco patterns enhance gameplay without distraction
- Zero visible lag between tile press and visual response
- Error feedback is clear but not punishing
- Game transitions are smooth and predictable

Reliability:

- Motors don't overheat during extended sequences
- WiFi AP handles reconnections gracefully
- System recovers from browser disconnect
- IR sensors work reliably in DDR mode

5. System Overview

The system consists of **four main components** working together:

1. **Browser Interface (HTML/JavaScript)** - Runs on laptop, displays game UI, contains all game logic (DDR, Memory, Whack-a-Mole, Tic-Tac-Toe), sends tile color commands, receives button presses

2. **Bridge Server (IDLE Python on laptop)** - HTTP proxy that polls both ESP32 boards, merges 9 floor buttons + 2 ceiling IR sensors into unified 11-element array, routes Neopixels colour and pattern commands to floor and state/event commands to ceiling

3. **Floor ESP32** - Hosts WiFi access point, runs HTTP server, reads 9 tile buttons, controls NeoPixels

4. **Ceiling ESP32** — Connects to floor's WiFi, runs HTTP server, controls 78-LED disco ring, 2 stepper motors, 2 lasers (always on), 2 IR sensors for DDR bonus detection

Communication flow:

```
Browser ←→ Bridge ←→ Floor ESP32 (buttons + LEDs)
                ↓
      Ceiling ESP32 (disco + steppers + IR)
```

All communication is HTTP-based. No WebSocket, no ESPNow. Bridge handles all polling and routing.

6. High-Level Block Diagram

7. Input / Output Mapping

Input	Process	Output
Tile button press	Floor GPIO detects → Bridge polls `/state` → Browser game logic	Tile color change via `/set_tile` → Floor NeoPixels update
Game state change	Browser detects new screen → Sends `/slave_msg?cmd=S:game:ddr` → Bridge forwards to ceiling	Ceiling state machine changes → Disco pattern + stepper sequence

IR beam break (DDR)	Ceiling GPIO detects → Sets sticky flag `ir_pressed[0]=1` → Bridge polls `/state`	Browser bonus handler executes → Sends `/slave_msg?cmd=E:bonus:left:on` → Stepper celebration spin
Wrong move	Browser game logic detects error → Sends `/slave_msg?cmd=E:wrong`	Ceiling triggers both steppers opposite directions (0.5s)
DDR speed increase	Browser tracks tempo → Sends `/slave_msg?cmd=D:1.5`	Ceiling multiplies disco scroll speed by 1.5×
TTT turn change	Browser alternates players → Sends `/slave_msg?cmd=S:game:ttt:pink`	Ceiling spins left stepper 1 rev (1.5s), disco changes to pink gradient
Win condition	Browser detects win → Sends `/slave_msg?cmd=S:end:win`	Ceiling plays celebration pattern (rainbow scroll + alternating spins)
Lose condition	Browser detects loss → Sends `/slave_msg?cmd=S:end:lose`	Ceiling plays failure pattern (red pulse + steppers stop)
Menu navigation	User steps LEFT/RIGHT → Floor button → Bridge → Browser	Browser updates menu → Sends `/slave_msg?cmd=S:menu:memory` → Disco changes to orange pattern
Game launch	User steps CENTER on menu → Browser confirms → Sends `/slave_msg?cmd=S:game:mole`	Ceiling transitions to game pattern, steppers stop

8. Sketches & Concepts

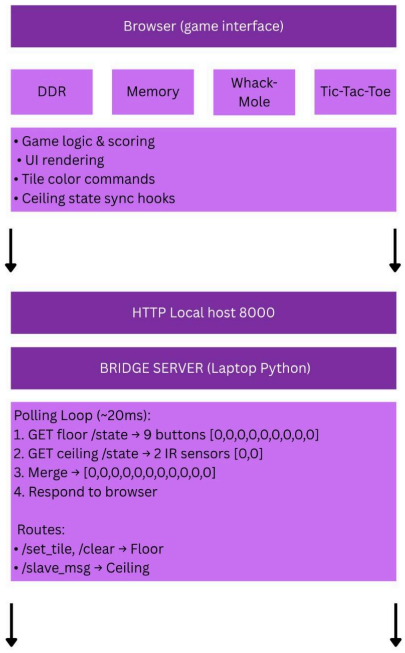
Early Concepts:

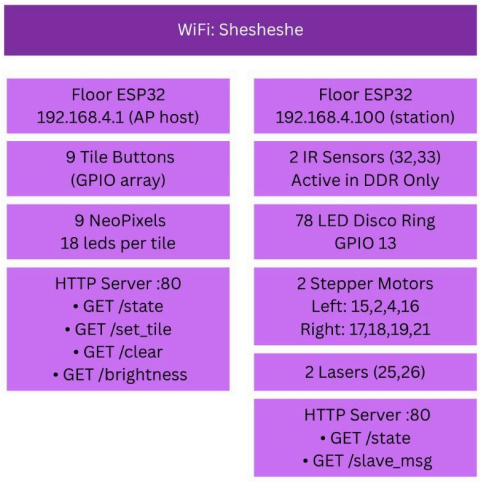
1. **Only DDR as game option**
2. **Original Architecture**
   - Master ESP32 ↔ ESPNow ↔ Slave ESP32
   - Browser ↔ WebSocket ↔ Master
   - **Problem**: ESPNow debugging difficult, WebSocket adds complexity
   - **Solution**: Bridge-centric HTTP — simpler, more reliable
3. **Laser Toggle Indicators**
   - Lasers originally turned on/off to indicate game events
   - **Problem**: Hardware didn't support GPIO connection

- **Solution**: Lasers always ON, rotation through steppers became primary indicators

4. **Center button choice lock in tic tac toe**
    - **Click center button to lock choice in tic tac toe**
    - **Problem**: unintuitive
    - **Solution**: double press the chosen tile

9. Final System Architecture





Power Requirements

FLOOR ESP32:
• ESP32 board: 5V 500mA
• NeoPixels (162 LEDs): 12V
• Recommended: 5V 3A power supply

CEILING ESP32:
• ESP32 board: 5V 500mA
• Disco ring (78 LEDs): 5V 1.5A peak
• Steppers (2×): 5V 500mA (via ULN2003 drivers)
• Lasers (2×): 5V 100mA
• Recommended: 5V 2.5A power supply

Boot Sequence

1. Run Bridge program in IDLE Python
→ HTTP server on http/localhost:8000

2. Power floor ESP32
   → WiFi AP "Shesheshe" starts

3. Power ceiling ESP32
   → Connects to "Shesheshe"
   → Gets static IP 192.168.4.100

4. Connect laptop to "Shesheshe"
   → Begins polling floor + ceiling

5. Open browser
   → Navigate to http://localhost:8000
   → Games load, system ready